

1. Vulnerability Report: Broken Root Detection Bypass

1. Description

The **BugBazaar** application employs a "Defense-in-Depth" root detection strategy to prevent execution on compromised devices. By restricting access on rooted devices, the app attempts to maintain a **Trusted Execution Environment** and prevent attackers from bypassing the Android sandbox to access private application data.

However, since these security checks occur on the client side, they are vulnerable to **Dynamic Instrumentation**. This allows an attacker to manipulate the application's runtime logic, forcing all security queries—even those in native C/C++ code—to return a "Safe" status.

2. Environment Preparation & Commands

Step 1: JADX Analysis (Identification)

- **Action:** Decompile the BugBazaar APK using **JADX-GUI**.
- **Findings:** Analysis revealed a complex detection matrix:
 - **Java Layer:** com.scottyab.rootbeer.RootBeer.
 - **Native Layer:** com.darwin.security.Native utilizing isMagiskPresentNative().
 - **IPC Layer:** An isolated service com.BugBazaar.IIsolatedService.

Step 2: Start Frida Server

- **Action:** Deploy the Frida server architecture matching the emulator (x86_64) to /data/local/tmp/.
- **Command:**
Bash
adb shell "chmod +x /data/local/tmp/frida-server-17.9.1-android-x86_64"
adb shell "/data/local/tmp/frida-server-17.9.1-android-x86_64 &"

Step 3: Implementation of Aggressive Bypass Script

- **Action:** A custom, full-spectrum JavaScript payload (rootbypass.js) was developed to hook Java methods, native system calls (libc), and Binder IPC transactions simultaneously.
- **Execution Command:**
PowerShell
frida -U -l rootbypass.js -f com.BugBazaar

3. Steps to Reproduce

1. **Baseline Test:** Launch BugBazaar on a rooted MEmu emulator. The application triggers the "This device is rooted" alert and terminates within 3 seconds.
2. **Instrumentation:** Execute the Frida command using the aggressive bypass script while the app is in a "Spawned" state.
3. **Full-Spectrum Hooking:** Observe the terminal logs as Frida intercepts multiple layers:
 - [+] Bypassed RootBeer.isRooted()
 - [+] Bypassed Native.isMagiskPresentNative()
 - [+] Blocked Binder onTransact code=1 (isMagiskPresent)
 - [NATIVE] Blocked access(): /system/xbin/su
4. **Final Verification:** The "Rooted" warning is suppressed, and the application successfully proceeds to the **Enter Passcode** screen.

4. Impact

- **Complete Defense Neutralization:** The application's primary barrier against runtime tampering is rendered ineffective.
- **Gateway to Further Exploitation:** Bypassing root detection is the "Master Key" that allows an attacker to:
 - Perform **SSL Unpinning** to intercept encrypted traffic.
 - Extract **Firebase API Keys** and **Razorpay** checkout parameters from memory.
 - Expose local databases containing sensitive user metadata.

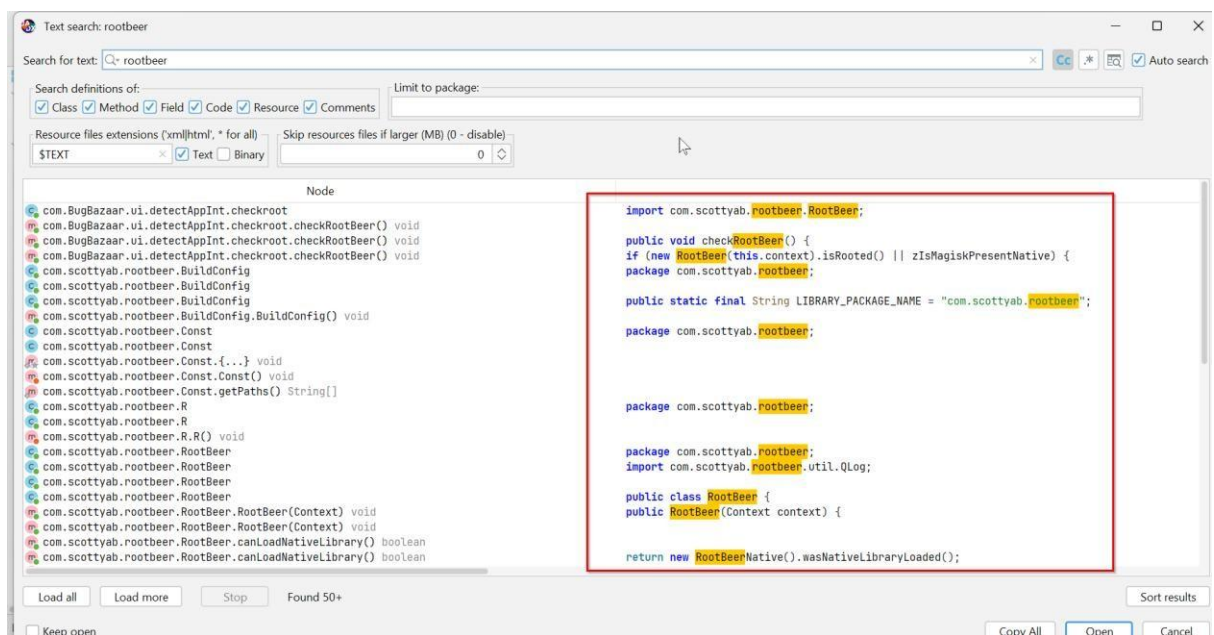
5. Severity

Medium/High

While root detection is a client-side check, its successful bypass in this application directly leads to the exposure of sensitive financial and cloud service integration points (Razorpay/Firebase).

6. Recommendations

1. **Google Play Integrity API:** Move away from local library checks. Use server-side attestation to verify device integrity before issuing session tokens.
2. **Native Hardening:** While this attack hooked `libc`, implementing detection in custom obfuscated C++ makes it significantly more time-consuming for an attacker to find the correct offsets.
3. **Anti-Frida Logic:** Implement checks to detect the presence of the Frida-server process or the `frida-gadget.so` library in memory to kill the session upon detection.



```
Command Prompt - frida --cc x Windows PowerShell x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\NO ESCAPE> adb shell
SM-N975F:/ # cd /data/local/tmp/
burp-der.crt oat/
frida-server-17.9.1-android-x86_64/
d /data/local/tmp/frida-server-17.9.1-android-x86_64/
/frida-server-17.9.1-android-x86_64
```

```
Java.perform(function() {
  console.log("[+] Loading Aggressive BugBazaar Bypass...");

  // ===== 1. RootBeer Java Layer =====
  try {
    var RootBeer = Java.use("com.scottyab.rootbeer.RootBeer");
    RootBeer.isRooted.implementation = function() {
      console.log("[+] Bypassed RootBeer.isRooted()");
      return false;
    };
  } catch(e) { console.log("[-] RootBeer not found: " + e); }

  // ===== 2. Native Magisk Detection (com.darwin.security) =====
  try {
    var NativeClass = Java.use("com.darwin.security.Native");
    NativeClass.isMagiskPresentNative.implementation = function() {
      console.log("[+] Bypassed Native.isMagiskPresentNative()");
      return false;
    };
  } catch(e) { console.log("[-] com.darwin.security.Native not found: " + e); }

  // ===== 3. AIDL Proxy Hook (IIolatedService) =====
  try {
    var AIDLProxy = Java.use("com.BugBazaar.IIolatedService$Stub$Proxy");
    AIDLProxy.isMagiskPresent.implementation = function() {
      console.log("[+] Bypassed AIDL IIolatedService.isMagiskPresent()");
      return false;
    };
  } catch(e) { console.log("[-] AIDL Proxy not found: " + e); }

  // ===== 4. AIDL Stub Hook (backup) =====
  try {
    var AIDLStub = Java.use("com.BugBazaar.IIolatedService$Stub");
    AIDLStub.isMagiskPresent.implementation = function() {
      console.log("[+] Bypassed AIDL Stub.isMagiskPresent()");
    };
  } catch(e) { console.log("[-] AIDL Stub not found: " + e); }
});
```

Ln 1, Col 1 9,625 characters Plain text 90% Windows (CRLF) UTF-8

```

Java.perform(function() {
    try {
        var AIDLStub = Java.use("com.BugBazaar.IIsolatedService$Stub");
        AIDLStub.isMagiskPresent.implementation = function() {
            console.log("[+] Bypassed AIDL Stub.isMagiskPresent()");
            return false;
        };
    } catch(e) { console.log("[-] AIDL Stub not found: " + e); }

    // ===== 5. Binder.onTransact() Low-level IPC Interception =====
    try {
        var Binder = Java.use('android.os.Binder');
        Binder.onTransact.overload('int', 'android.os.Parcel', 'android.os.Parcel', 'int').implementation = function(code, data, reply, flags) {
            // Transaction code 1 is often isMagiskPresent - block it
            if (code == 1) {
                console.log("[+] Blocked Binder onTransact code=" + code + " (isMagiskPresent)");
                reply.writeNoException();
                reply.writeInt(0);
                return true;
            }
            return this.onTransact(code, data, reply, flags);
        };
    } catch(e) { console.log("[-] Binder.onTransact hook failed: " + e); }

    // ===== 6. File.exists checks for su/magisk paths =====
    try {
        var File = Java.use("java.io.File");
        File.exists.implementation = function() {
            var path = this.getPath();
            if (path.indexOf("su") >= 0 || path.indexOf("magisk") >= 0 ||
                path.indexOf("busybox") >= 0 || path.indexOf("supersu") >= 0) {
                console.log("[+] Blocked File.exists() on: " + path);
                return false;
            }
            return this.exists();
        };
    } catch(e) { console.log("[-] File.exists hook failed: " + e); }
});

```

```

        return this.exists();
    } catch(e) { console.log("[-] File.exists hook failed: " + e); }

    // ===== 7. Runtime.exec block for root commands =====
    try {
        var Runtime = Java.use("java.lang.Runtime");
        Runtime.exec.overload('[Ljava.lang.String;').implementation = function(cmdarray) {
            var cmdStr = cmdarray.join(" ");
            if (cmdStr.indexOf("su") >= 0 || cmdStr.indexOf("magisk") >= 0 ||
                cmdStr.indexOf("id") >= 0 || cmdStr.indexOf("which") >= 0) {
                console.log("[+] Blocked Runtime.exec: " + cmdStr);
                return null;
            }
            return this.exec(cmdarray);
        };
        Runtime.exec.overload('java.lang.String').implementation = function(cmd) {
            if (cmd.indexOf("su") >= 0 || cmd.indexOf("magisk") >= 0 ||
                cmd.indexOf("id") >= 0 || cmd.indexOf("which") >= 0) {
                console.log("[+] Blocked Runtime.exec: " + cmd);
                return null;
            }
            return this.exec(cmd);
        };
    } catch(e) { console.log("[-] Runtime.exec hook failed: " + e); }

    // ===== 8. ProcessBuilder block =====
    try {
        var ProcessBuilder = Java.use("java.lang.ProcessBuilder");
        ProcessBuilder.start.implementation = function() {
            var cmd = this.command();
            if (cmd) {
                var cmdStr = JSON.stringify(cmd);
                if (cmdStr.indexOf("su") >= 0 || cmdStr.indexOf("magisk") >= 0) {
                    console.log("[+] Blocked ProcessBuilder.start: " + cmdStr);
                    return null;
                }
            }
            return this.start();
        };
    } catch(e) { console.log("[-] ProcessBuilder.start hook failed: " + e); }
});

```

```

Java.perform(function() {
    var ProcessBuilder = Java.use("java.lang.ProcessBuilder");
    ProcessBuilder.start.implementation = function() {
        var cmd = this.command();
        if (cmd) {
            var cmdStr = JSON.stringify(cmd);
            if (cmdStr.indexOf("su") >= 0 || cmdStr.indexOf("magisk") >= 0) {
                console.log("[+] Blocked ProcessBuilder.start: " + cmdStr);
                return null;
            }
        }
        return this.start();
    };
} catch(e) { console.log("[~] ProcessBuilder hook failed: " + e); }

// ===== 9. SystemProperties spoof =====
try {
    var SystemProperties = Java.use("android.os.SystemProperties");
    SystemProperties.get.overload('java.lang.String').implementation = function(key) {
        if (key.indexOf("ro.debuggable") >= 0) {
            console.log("[+] Spoofed ro.debuggable -> 0");
            return "0";
        }
        if (key.indexOf("ro.secure") >= 0) {
            console.log("[+] Spoofed ro.secure -> 1");
            return "1";
        }
        if (key.indexOf("ro.build.tags") >= 0) {
            console.log("[+] Spoofed ro.build.tags -> release-keys");
            return "release-keys";
        }
        if (key.indexOf("ro.build.type") >= 0) {
            console.log("[+] Spoofed ro.build.type -> user");
            return "user";
        }
        return this.get(key);
    };
} catch(e) { console.log("[~] SystemProperties hook failed: " + e); }

// ===== 10. Build spoof (emulator detection) =====
try {
    var Build = Java.use("android.os.Build");
    Build.FINGERPRINT.value = "samsung/crownqltesq/crownqltesq:12/SP1A.210812.016/G965USQU8EVG3:user/release-keys";
    Build.MODEL.value = "SM-G965U";
    Build.MANUFACTURER.value = "samsung";
    Build.BRAND.value = "samsung";
    Build.DEVICE.value = "crownqltesq";
    Build.PRODUCT.value = "crownqltesq";
    Build.HARDWARE.value = "qcom";
    console.log("[+] Spoofed Build properties to Samsung device");
} catch(e) { console.log("[~] Build spoof failed: " + e); }

// ===== 11. Debug flag hook =====
try {
    var Debug = Java.use("android.os.Debug");
    Debug.isDebugEnabledConnected.implementation = function() {
        console.log("[+] Blocked Debug.isDebugEnabledConnected");
        return false;
    };
} catch(e) { console.log("[~] Debug hook failed: " + e); }

console.log("[✓] Aggressive bypass loaded - all hooks active");
});

// ===== NATIVE LAYER: Hook access() and fopen() for su/magisk =====
// This blocks native-level file existence checks in libc
var nativeHooksPatched = false;

```

```

Java.perform(function() {
    if (key.indexOf("ro.build.type") >= 0) {
        console.log("[+] Spoofed ro.build.type -> user");
        return "user";
    }
    return this.get(key);
} catch(e) { console.log("[~] SystemProperties hook failed: " + e); }

// ===== 10. Build spoof (emulator detection) =====
try {
    var Build = Java.use("android.os.Build");
    Build.FINGERPRINT.value = "samsung/crownqltesq/crownqltesq:12/SP1A.210812.016/G965USQU8EVG3:user/release-keys";
    Build.MODEL.value = "SM-G965U";
    Build.MANUFACTURER.value = "samsung";
    Build.BRAND.value = "samsung";
    Build.DEVICE.value = "crownqltesq";
    Build.PRODUCT.value = "crownqltesq";
    Build.HARDWARE.value = "qcom";
    console.log("[+] Spoofed Build properties to Samsung device");
} catch(e) { console.log("[~] Build spoof failed: " + e); }

// ===== 11. Debug flag hook =====
try {
    var Debug = Java.use("android.os.Debug");
    Debug.isDebugEnabledConnected.implementation = function() {
        console.log("[+] Blocked Debug.isDebugEnabledConnected");
        return false;
    };
} catch(e) { console.log("[~] Debug hook failed: " + e); }

console.log("[✓] Aggressive bypass loaded - all hooks active");
});

// ===== NATIVE LAYER: Hook access() and fopen() for su/magisk =====
// This blocks native-level file existence checks in libc
var nativeHooksPatched = false;

```

```

Java.perform(function() {
  function hookNativeLibc(moduleName) {
    try {
      var libc = Module.findExportByName(moduleName, "access");
      if (libc) {
        Interceptor.attach(libc, {
          onEnter: function(args) {
            var path = Memory.readCString(args[0]);
            if (path && (path.indexOf("su") >= 0 || path.indexOf("magisk") >= 0 ||
              path.indexOf("busybox") >= 0 || path.indexOf(".ext") >= 0)) {
              console.log("[NATIVE] Blocked access(): " + path);
              args[0] = Memory.allocUtf8String("/nonexistent");
            }
          }
        });
      }

      var fopen = Module.findExportByName(moduleName, "fopen");
      if (fopen) {
        Interceptor.attach(fopen, {
          onEnter: function(args) {
            var path = Memory.readCString(args[0]);
            if (path && (path.indexOf("su") >= 0 || path.indexOf("magisk") >= 0)) {
              console.log("[NATIVE] Blocked fopen(): " + path);
              args[0] = Memory.allocUtf8String("/nonexistent");
            }
          }
        });
      }

      var stat = Module.findExportByName(moduleName, "stat");
      if (stat) {
        Interceptor.attach(stat, {
          onEnter: function(args) {
            var path = Memory.readCString(args[0]);
            if (path && (path.indexOf("su") >= 0 || path.indexOf("magisk") >= 0)) {
              console.log("[NATIVE] Blocked stat(): " + path);
            }
          }
        });
      }
    }
  }
});

```

```

    onEnter: function(args) {
      var path = Memory.readCString(args[0]);
      if (path && (path.indexOf("su") >= 0 || path.indexOf("magisk") >= 0)) {
        console.log("[NATIVE] Blocked stat(): " + path);
        args[0] = Memory.allocUtf8String("/nonexistent");
      }
    }
  });
}

var openat = Module.findExportByName(moduleName, "openat");
if (openat) {
  Interceptor.attach(openat, {
    onEnter: function(args) {
      var path = Memory.readCString(args[1]);
      if (path && (path.indexOf("su") >= 0 || path.indexOf("magisk") >= 0)) {
        console.log("[NATIVE] Blocked openat(): " + path);
        args[1] = Memory.allocUtf8String("/nonexistent");
      }
    }
  });
}
} catch(e) { console.log("[NATIVE] Hook error on " + moduleName + ": " + e); }
}

// Hook across all loaded modules that contain libc
setTimeout(function() {
  hookNativeLibc("libc.so");
  hookNativeLibc("libc++.so");

  // Also try hooking the app's native library directly
  var nativeLib = Process.findModuleByName("native-lib.so");
  if (nativeLib) {
    console.log("[NATIVE] Found native-lib.so at base: " + nativeLib.base);
  }
}, 500);

```


8:22

Enter Passcode

1 2 3

4 5 6

7 8 9

0

VALIDATE

SSL Pinning is a security mechanism where an application "pins" the expected server certificate or public key. This ensures the app only communicates with the legitimate server, even if the device trusts a rogue Certificate Authority (CA). By bypassing this, an attacker can perform a **Man-in-the-Middle (MITM)** attack to capture sensitive data such as API keys, user credentials, and payment details.

2. Environment Preparation & Commands

Step 1: Frida Server Setup

The Frida server must be running on the mobile device to allow dynamic instrumentation.

- **Command:**
Bash
adb shell
`cd /data/local/tmp/`
`./frida-server-17.9.1-android-x86_64 &`

Step 2: Burp Suite Configuration

- Configure Burp Suite to listen on all interfaces.
- Export and install the Burp CA certificate on the Android device as a **System CA** (or use a script to force trust).

Step 3: Execute SSL Pinning Bypass

- **Screenshot Reference:** image_cf4b14.png and image_cf4af7.png
- **Command:**
Bash
`frida --codeshare akabel/frida-multiple-unpinning -f com.BugBazaar -U`
(This script targets multiple common Android libraries like OkHttp, TrustManagerImpl, and WebViews.)

3. Steps to Reproduce

1. Set the Android device proxy to point to your **Burp Suite** instance.
2. Launch the application using the Frida command above.
3. Observe the terminal output: You will see Frida searching for pinning methods and successfully applying hooks.
 - o **Screenshot Reference:** image_cf4af7.png shows: [+] Bypassing TrustManagerImpl (Android > 7) checkTrustedRecursive.
4. Navigate through the app (e.g., go to the checkout or profile section).
5. Check **Burp Suite (HTTP History)**: Encrypted traffic that was previously hidden is now visible in cleartext. o **Screenshot Reference:** {2D76E239-870D-4B39-8D78-F148430F2B84}.png shows captured Firebase and Razorpay requests.

4. Impact

- **Sensitive Data Exposure:** Intercepting traffic reveals internal API endpoints, authentication tokens, and PII (Personally Identifiable Information).
- **API Exploitation:** Once the traffic is visible, an attacker can move to server-side attacks like **IDOR**, **SQL Injection**, or **Price Manipulation**.
- **Loss of Confidentiality:** All communications between the user and the server are no longer private.
-

5. Severity

High (SSL Pinning is a critical defense-in-depth layer. Bypassing it is the gateway to most other server-side vulnerabilities in mobile pentesting.)

6. Recommendation

1. **Obfuscation:** Use **ProGuard** or **DexGuard** to obfuscate the names of network classes and methods. This makes it significantly harder for universal Frida scripts to find the correct "hooks."
2. **Native Implementation:** Implement SSL pinning in the **Native (C/C++) layer** using **OpenSSL** or **BoringSSL**. Native hooks are much more complex to write than Java-level hooks.
3. **Certificate Transparency:** Implement Certificate Transparency (CT) logging to detect if a certificate has been maliciously issued.
4. **Hardware-Backed Security:** Use the Android **KeyStore** to store sensitive certificates, ensuring they cannot be easily extracted or replaced in memory.

```
Windows PowerShell x Windows PowerShell x + v
1246 Google Play Store com.android.vending
1002 MEMu Launcher2 com.microvirt.launcher2
- Allsafe infosecadventures.allsafe
- BugBazaar com.BugBazaar
- Chrome com.android.chrome
- File Manager com.cyanogenmod.filemanager
- Files com.android.documentsui
- Gallery com.android.gallery3d
- MEMu Guide com.microvirt.guide
- Settings com.android.settings
PS C:\Users\NO ESCAPE> frida --codeshare pcipolloni/universal-android-ssl-pinning-bypass-with-frida -f com.BugBazaar -U

/-----| Frida 17.9.1 - A world-class dynamic instrumentation toolkit
|  C  |
|_ _ _|
|_ _ _| Commands:
|_ _ _| help -> Displays the help system
|_ _ _| object? -> Display information about 'object'
|_ _ _| exit/quit -> Exit
|_ _ _|
|_ _ _| More info at https://frida.re/docs/home/
|_ _ _|
|_ _ _| Connected to SM N975F (id=127.0.0.1:21503)
Spawned 'com.BugBazaar'. Resuming main thread!
[SM N975F::com.BugBazaar ]->
[.] Cert Pinning Bypass/Re-Pinning
[+] Loading our CA...
[o] Our CA Info: CN=PortSwigger CA, OU=PortSwigger CA, O=PortSwigger, L=PortSwigger, ST=PortSwigger, C=PortSwigger
[+] Creating a KeyStore for our CA...
[+] Creating a TrustManager that trusts the CA in our KeyStore...
[+] Our TrustManager is ready...
[+] Hijacking SSLContext methods now...
[-] Waiting for the app to invoke SSLContext.init()...
```

```

Windows PowerShell
Exiting!

Thank you for using Frida!
PS C:\Users\NO ESCAPE> frida --codeshare akabel/frida-multiple-unpinning -f com.BugBazaar -U

  /---\
 | C |   Frida 17.9.1 - A world-class dynamic instrumentation toolkit
 |___|
  >_ _ _
 /_/_/_\ Commands:
         help      -> Displays the help system
         object?   -> Display information about 'object'
         exit/quit -> Exit
         . . . . .
         . . . . . More info at https://frida.re/docs/home/
         . . . . .
         . . . . . Connected to SM N975F (id=127.0.0.1:21503)
Spawning 'com.BugBazaar'...
Hello! This is the first time you're running this particular snippet, or the snippet's source code has changed.

Project Name: frida-multiple-unpinning
Author: @akabel
Slug: akabel/frida-multiple-unpinning
Fingerprint: 1a854b037b6b46dab29466c9aa055db5aed4734c8c6626177bfcd8b1a87311
URL: https://codeshare.frida.re/@akabel/frida-multiple-unpinning

Are you sure you'd like to trust this project? [y/N] y
Adding fingerprint 1a854b037b6b46dab29466c9aa055db5aed4734c8c6626177bfcd8b1a87311 to the trust store! You won't be prompted again unless the code changes.
Spawned 'com.BugBazaar'. Resuming main thread!
[SM N975F::com.BugBazaar ]->
=====
[#] Android Bypass for various Certificate Pinning methods [#]
=====
[-] OkHTTPv3 {2} pinner not found

```

```

Windows PowerShell
[-] GMS TrustManagerImpl (Android > 7) checkTrustedRecursive {1} check not found
[-] GMS TrustManagerImpl (Android > 7) checkTrustedRecursive {2} check not found
[-] GMS TrustManagerImpl (Android > 7) verifyChain {1} check not found
[-] GMS TrustManagerImpl (Android > 7) verifyChain {2} check not found
[-] Appcelerator PinningTrustManager pinner not found
[-] Fabric PinningTrustManager pinner not found
[-] OpenSSLSocketImpl Conscrypt {1} pinner not found
[-] OpenSSLSocketImpl Conscrypt {2} pinner not found
[-] OpenSSLEngineSocketImpl Conscrypt pinner not found
[-] OpenSSLSocketImpl Apache Harmony pinner not found
[-] PhoneGap sslCertificateChecker pinner not found
[-] IBM MobileFirst pinTrustedCertificatePublicKey {1} pinner not found
[-] IBM MobileFirst pinTrustedCertificatePublicKey {2} pinner not found
[-] IBM WorkLight HostNameVerifierWithCertificatePinning {1} pinner not found
[-] IBM WorkLight HostNameVerifierWithCertificatePinning {2} pinner not found
[-] IBM WorkLight HostNameVerifierWithCertificatePinning {3} pinner not found
[-] IBM WorkLight HostNameVerifierWithCertificatePinning {4} pinner not found
[-] Conscrypt CertPinManager (Legacy) pinner not found
[-] CWAC-Netsecurity CertPinManager pinner not found
[-] Worklight Androidgap WLCertificatePinningPlugin pinner not found
[-] Netty FingerprintTrustManagerFactory pinner not found
[-] Squareup CertificatePinner {1} pinner not found
[-] Squareup CertificatePinner {2} pinner not found
[-] Squareup OkHostnameVerifier check not found
[-] Squareup OkHostnameVerifier check not found
[-] Android WebViewClient {3} check not found
[-] Apache Cordova WebViewClient check not found
[-] Boye AbstractVerifier check not found
[-] Chromium Cronet pinner not found
[-] Flutter HttpCertificatePinning pinner not found
[-] Flutter SslPinningPlugin pinner not found
[+] Bypassing TrustManagerImpl (Android > 7) checkTrustedRecursive {2} check: null firebaseinappmessaging.googleapis.com

```

